

After running this script in guru mode, whenever you try to create a directory with the name “mydir”, you will get an ‘Invalid argument’ error.

Tracing for custom modules

Now, let’s suppose you want to trace functions from custom modules that you have written. For instance: shown below are *module1* and *module2*. *Module1* exports a function which *module2* uses:

```
=====module1=====
#include <linux/module.h>

static int __init my_init( void )
{
    printk("Hello world from module 1 \n");
    return 0;
}

static void __exit my_exit( void )
{
    printk("Goodbye world from module 1 \n");
}

static void modifun(void)
{
    printk(" YAHOO ! I got into modifun \n");
}

EXPORT_SYMBOL(modifun);

module_init( my_init );
module_exit( my_exit );
MODULE_LICENSE("GPL");

=====module2=====
#include <linux/module.h>
extern void modifun(void);

static int __init my_init( void )
{
    printk("Hello world from module2\n");
    modifun();
    return 0;
}

static void __exit my_exit( void )
{
    printk("Goodbye world from module2\n");
}

module_init( my_init );
module_exit( my_exit );
MODULE_LICENSE("GPL");
```

After compiling both modules, I have inserted *module1*. Now, to probe whenever *mod1fun* is called, I run the following script:

```
probe module("module1").function("") {
    printf("%s was called from somewhere \n", probefunc());
}

Pass 1: parsed user script and 50 library script(s) in 240usr/30sys/271real
ms.
semantic error: missing x86_64 kernel/module debuginfo under '/
lib/modules/<kernel version>/build' while resolving probe point
module("module1").function("")
Pass 2: analyzed script: 0 probe(s), 0 function(s), 0 embed(s), 0 global(s)
in 90usr/300sys/474real ms.
Pass 2: analysis failed. Try again with another '-vp 81' option.
...
```

As the output shows, *stap* tries to search for information about *module1* under */lib/modules/<kernel version>/build*, but doesn’t find it, and gives an error. Let’s create a directory under */lib/modules/<kernel version>*, and copy *module1.ko* there:

```
$ mkdir /lib/modules/<kernel version> /custom
$ cp <Module_Path>/module1.ko /lib/modules/<kernel version> /custom/
```

Now, let’s run the *stap* script again. Then, as you load *module2*, you should see the following output in the terminal in which the script was running:

```
modifun was called from somewhere
```

In this manner, you can trace any custom modules; you just need to have *debuginfo* installed for those modules where *stap* can find it in Pass 2. The possibilities are endless. There are a lot of other interesting things you can do. I would recommend that you go through the references mentioned below, and try out as many examples as you can from the SystemTap website. 

References

<http://sourceware.org/systemtap/>
<http://sourceware.org/systemtap/wiki/LV2008SystemTapTutorial>
<http://sourceware.org/systemtap/wiki/HomePage?action=AttachFile&do=view&target=fosdem-stap.pdf>
<http://www.redbooks.ibm.com/redpapers/pdfs/redp4469.p>

By: Neependra Khare

The author is an open source enthusiast with a deep interest in Linux. He is currently working as a software engineer at KQ Infotech, and can be reached at neependra.khare@gmail.com. He also provides training on the Linux kernel and on debugging tools.